Evaluating Raft-Based Resilience Mechanisms for Containerized Distributed Key-Value Stores Under Realistic Fault Scenarios

Maria-Miruna Stoinea Ana-Maria Ungureanu Andra-Ștefania Popoiu
Faculty of Computer Science
Alexandru Ioan Cuza University of Iași

***Abstract*—Distributed systems must provide high availability and strong data consistency even under node crashes, network partitions, and transient connectivity loss. This paper presents the design, implementation, and systematic experimental evaluation of a containerized three-node key-value store built on a simplified variant of the Raft consensus algorithm. We contribute: (i) a fault-injection testbed implemented in Python and Docker Compose; (ii) six reproducible experiments covering normal operation, leader crash, network partition, majority loss, baseline latency comparison, and automated remediation; and (iii) quantitative evidence for five resilience properties. Under normal operation the system achieves 100% write availability at 11.21 req/s with a mean write latency of 38.8 ms. Quorum replication imposes a measurable overhead of 14.3 ms on average (33.0 ms at p99) over single-node reads. Leader re-election completes within the configured 3–6 s timeout window; split-brain writes are provably blocked by the quorum mechanism; and an automated watchdog restores failed containers within 2 s. All results are accompanied by an integrated requirements and conformance table linking each system property to its verifying experiment.**

***Index Terms*—distributed systems, Raft consensus, fault tolerance, leader election, quorum replication, chaos engineering, automated remediation, key-value store, containerization, Docker**

I. Introduction

Modern distributed systems underpin critical infrastructure (cloud platforms, financial services, healthcare records, and communication networks), where unplanned downtime carries severe economic and operational consequences. The fundamental tension in such systems is captured by the CAP theorem [3], [4]: a network-partitioned system cannot simultaneously guarantee both strong consistency and high availability. Designing for resilience therefore requires an explicit trade-off policy, a verified recovery mechanism, and empirical evidence that the policy holds under realistic fault conditions.

The Raft consensus algorithm [1] was introduced as an understandable alternative to Paxos [2] for managing replicated state machines. Raft decomposes consensus into three largely independent sub-problems: leader election, log replication, and safety. It provides a formal model that is easier to implement correctly and reason about. Production systems such as *etcd* [12] (the Kubernetes configuration store) and CockroachDB rely on Raft derivatives; understanding the empirical cost and behavior of Raft under failure is therefore of direct practical relevance.

This paper addresses the following research question: *Which resilience mechanisms suffice to maintain correct and available operation of a distributed key-value store under realistic single- and multi-node failure scenarios, and what quantitative cost does consensus impose?*

Our contributions are:

- A containerized three-node Raft testbed with quorum-based writes, data persistence, and a REST key-value API, deployable with a single `docker-compose up` command;
- Six fault-injection experiments with reproducible scripts and measured outputs;
- A quantitative baseline comparing read latency (no consensus) against quorum write latency at p50, p95, and p99 percentiles;
- A message-overhead metric demonstrating 2.18 internal replication messages per committed write;
- An automated watchdog extension demonstrating sub-2-second container self-healing;
- An integrated system requirements and conformance specification table (12 requirements, 5 conformance checks).

The remainder of this paper is organized as follows. Section II reviews related work. Section III formalizes the problem. Sections IV and V describe system design and implementation. Sections VI and VII present the experimental methodology and results. Section VIII gives the conformance specification. Sections IX–XII provide discussion, security analysis, limitations, and conclusions.

II. Background and Related Work

Understanding the guarantees and limits of distributed consensus requires grounding in three foundational results: the CAP theorem, the FLP impossibility, and the Raft algorithm itself. We also situate our work among comparable production systems and describe the chaos engineering methodology that guides our fault-injection design.

A. CAP Theorem and Consistency Models

Brewer's CAP conjecture [3], formalized by Gilbert and Lynch [4], states that a distributed system can guarantee at most two of: Consistency, Availability, and Partition tolerance. Since network partitions are unavoidable in practice, system

designers must choose between CP (consistent but potentially unavailable) and AP (available but potentially inconsistent). Raft systems are CP: under partition they sacrifice availability rather than risk data divergence. Our majority-loss experiment (Section VII) explicitly demonstrates this behavior by showing that a sub-quorum node refuses to commit writes.

B. FLP Impossibility

Fischer, Lynch, and Paterson [5] proved that no deterministic protocol can guarantee consensus in an asynchronous system with even one faulty process. Practical systems circumvent this impossibility by introducing probabilistic timing assumptions (timeouts). Raft's randomized election timeout, sampled uniformly from a configurable interval, ensures that at most one candidate wins an election in any given time window with high probability.

C. Raft Consensus

Ongaro and Ousterhout [1] designed Raft to be comprehensible without sacrificing correctness. Raft guarantees: *Election Safety* (at most one leader per term), *Leader Append-Only* (a leader never overwrites its log), *Log Matching* (logs with equal index and term are identical), *Leader Completeness* (committed entries are present in all future leaders), and *State Machine Safety* (servers apply the same commands in the same order). Our implementation covers leader election and heartbeat-based state replication; Section IV details the design choices.

D. Related Systems

ZooKeeper [6] uses the ZAB (ZooKeeper Atomic Broadcast) protocol for coordination. Like Raft, it is a CP system with a single-writer model, used extensively for distributed configuration management. Spanner [7] uses Multi-Paxos with TrueTime GPS clocks, achieving global ACID transactions at significant infrastructure cost. Chandra and Toueg [9] introduced the failure-detector abstraction underpinning many modern consensus protocols; Raft's heartbeat timeout can be understood as an implementation of the $\Diamond W$ (eventually weak) failure detector.

E. Chaos Engineering

Chaos engineering [10] is the discipline of deliberately injecting faults into production-like systems to surface hidden weaknesses. Our experiment suite uses Docker container termination (`docker stop`) and network disconnection (`docker network disconnect`), following the chaos engineering principle of controlled, observable, minimal-blast-radius experiments. Kleppmann [11] provides a comprehensive treatment of the failure modes our testbed targets, including split-brain, leader election, and quorum semantics.

III. Problem Formulation

Research question. Can a three-node Raft cluster implemented in a lightweight containerized environment satisfy strong consistency (CP) while providing measurable availability guarantees under node crashes, network partitions, and majority-node failures?

Failure model. We consider three classes of faults:

1) *Crash-stop failures*: a node halts and does not resume until externally restarted;
2) *Network partitions*: a node loses bidirectional connectivity to all peers while remaining locally functional;
3) *Majority failures*: two of three nodes crash simultaneously, making quorum formation impossible.

Byzantine faults (nodes sending malicious or inconsistent messages) are explicitly out of scope.

Consistency model. The system implements *strong consistency for writes*: all writes accepted by the leader are replicated to a quorum ($\lceil n/2 \rceil + 1 = 2$ for $n = 3$) before acknowledgement. Reads are served by any replica from its local state (read-your-writes for local reads; eventual read consistency across replicas).

Metrics. Following the project specification [14], we measure: write availability (percentage), recovery time after leader crash, consensus replication overhead, message overhead per write, time to split-brain resolution, and time to automated remediation.

IV. System Design

The testbed is organized into four concerns: cluster topology and inter-node communication, the Raft protocol state machine, data persistence, and external monitoring. Each is described in a dedicated subsection below.

A. Architecture Overview

The testbed consists of three identical Raft nodes deployed as Docker containers connected by a bridged Docker network (`raft_network`). Each node exposes a Flask REST API on a dedicated host-mapped port (5001–5003) and maintains a local key-value store replicated via the Raft protocol. An external watchdog process monitors container health and automatically restarts failed containers. Fig. 1 shows the full system architecture.

B. Raft Protocol Design

Each node is a state machine with three states: FOLLOWER, CANDIDATE, and LEADER. Table I lists the key configuration parameters.

TABLE I
RAFT CONFIGURATION PARAMETERS

Parameter	Value
Election timeout	Uniform $[3.0, 6.0]$ s
Heartbeat interval	1.0 s
Quorum size	2 of 3 nodes
Replication RPC timeout	400 ms
Vote request timeout	400 ms
Cluster size	3 nodes

Fig. 2 shows the complete node state machine with all transitions and their triggering conditions.

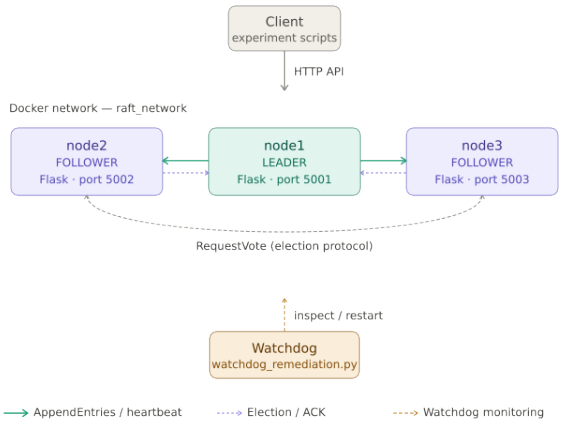

Fig. 1. System architecture. The center node acts as LEADER (teal); the other two are FOLLOWERs (purple). AppendEntries RPCs propagate writes; an external Watchdog monitors and auto-restarts containers via `docker inspect/start`. Client scripts communicate via HTTP REST.

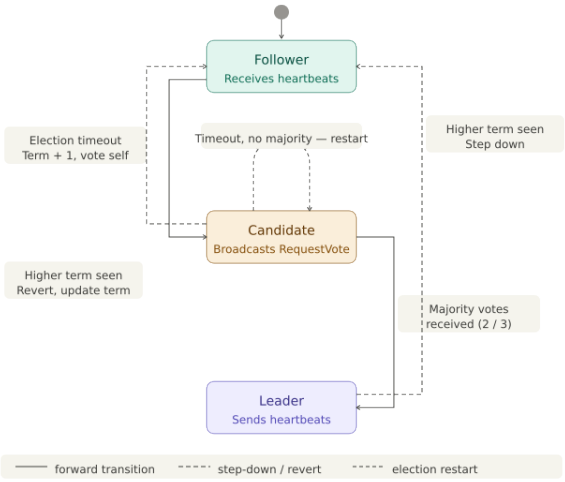

Fig. 2. Raft node state machine. Solid arrows indicate forward transitions; dashed arrows indicate step-down or revert transitions. A follower transitions to candidate on election timeout, incrementing its term and voting for itself. A candidate winning a strict majority (2/3) advances to leader. Any node receiving a message carrying a higher term reverts immediately to follower. A candidate that times out without achieving majority restarts the election with an incremented term.

Leader election. On election timeout, a follower increments its term, transitions to Candidate, votes for itself, and broadcasts `RequestVote` RPCs. A node receiving a `RequestVote` with a strictly higher term immediately converts to follower, updates its term, and grants its vote; subsequent candidates in the same term are denied (one vote per term per node).

Quorum-based write replication. Write requests (`PUT`) are only accepted by the leader. The leader replicates the updated store state to all followers in parallel via `AppendEntries` RPCs. The write is committed and acknowledged to the client only after at least 2 of 3 responses are received. This guarantees that any committed value is recoverable from at least one surviving node after a single-node failure.

Step-down on stale term. When processing heartbeat responses from peers, if any response carries a term higher than the leader's current term, the leader immediately reverts to follower state and updates its term. This mechanism resolves split-brain scenarios: upon network healing, the isolated old leader receives a higher-term heartbeat and steps down without manual intervention.

C. Data Persistence

Committed key-value pairs are serialized to a local JSON file (`/tmp/raft_store_{node_id}.json`) immediately after quorum validation. On startup, each node loads this file before participating in elections. This provides crash recovery for single-node restarts. Full Write-Ahead Log (WAL) semantics remain a limitation, discussed in Section XI.

D. Message Overhead Instrumentation

A thread-safe atomic counter tracks every outbound `AppendEntries` RPC (both heartbeat and write-replication). The counter is exposed via the `/status` endpoint and sampled before and after benchmark runs to derive per-write message overhead.

E. Automated Remediation (Watchdog)

An external watchdog process polls container status every 2 s via `docker inspect`. On detecting an `exited` container, it issues `docker start` and logs the event. This component implements the chaos-engineering automated remediation extension specified in the project requirements [14].

V. Implementation

The implementation totals over 800 lines of Python across eight source files. The core node logic (`src/node.py`, ≈240 lines) uses Flask 2.3.2 for the HTTP layer and Python's `threading` module for concurrent election, heartbeat, and replication threads. Fig. 3 illustrates the complete message flow for a quorum-committed write.

To demonstrate the synchronous quorum-backed write pipeline, Listing 1 details the critical section from `node.py` where parallel replication threads are joined and evaluated against the strict majority threshold.

Full-state replication. Rather than maintaining an indexed log of individual entries as specified in the original Raft paper [1], the leader transmits a complete key-value store snapshot in each AppendEntries RPC. This simplification preserves the correctness properties needed for our experiments (leader election, quorum enforcement, and term-based step-down) while eliminating log compaction and snapshot-transfer complexity. The trade-off ($O(|\text{store}|)$ bandwidth per heartbeat) is acceptable for the small synthetic workloads used in this study and is explicitly acknowledged in Section XI.

Containerization and fault injection. Each node runs in an identical Docker image based on `python:3.9-slim`. Docker Compose orchestrates the cluster. Fault injection uses `docker stop` (crash fault), `docker network`

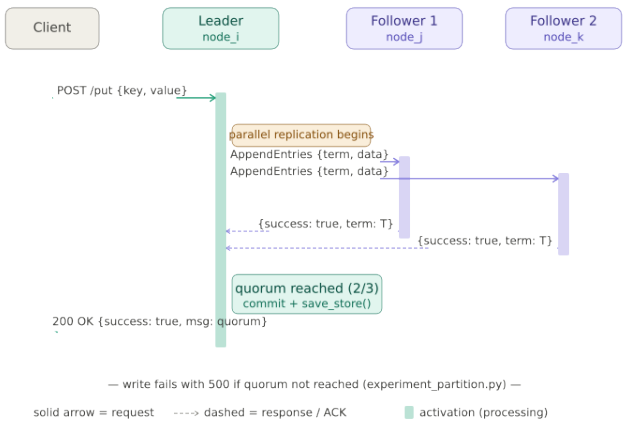

Fig. 3. Sequence diagram for a quorum write operation. The leader replicates to both followers in parallel; the client receives 200 OK only after $\geq$2 ACKs confirm the commit. A write to an isolated leader returns a timeout/500 error (shown in experiment E4).

Listing 1. Synchronous Quorum Validation Logic

```
# Wait for parallel replication threads to finish (max 400
    ms timeout)
for t in threads:
    t.join()

# Strict majority check (N/2 + 1)
quorum_needed = (len(ALL_NODES) // 2) + 1

if sum(replication_successes) >= quorum_needed:
    store[key] = value # Commit locally
    save_store()       # Persist to disk
    return jsonify({'success': True, 'msg': 'Data
        replicated to quorum', 'store_size': len(store)})
else:
    return jsonify({'error': 'Write failed. Quorum could
        not be reached.'}), 500
```

`disconnect/connect` (partition fault), and `docker start` (recovery), avoiding any modification to application code and ensuring that experiments are reproducible independently of the application logic.

REST API. Each node exposes: `POST /put` (write, leader only), `GET /get` (read, any node), `GET /status` (node state, term, leader ID, store size, message count), `POST /append_entries` (internal), `POST /request_vote` (internal).

VI. Experimental Methodology

All experiments were executed on a single-host Docker Compose cluster and are fully reproducible from the repository root. We describe the platform, the synthetic workload, and the six fault scenarios in the subsections below.

A. Platform

All experiments were conducted on a Windows 11 host running Docker Desktop. The three containers share a single physical machine and communicate over a Docker bridge network with sub-millisecond inter-container round-trip time. Results represent a best-case latency scenario; geographically distributed deployments would exhibit higher and more variable latencies.

B. Workload and Dataset

The synthetic write workload consists of 100 sequential PUT requests with keys `key_0...key_99` and values `val_0...val_99` at 50 ms inter-arrival intervals. The baseline comparison uses 50 GET requests followed by 50 PUT requests with distinct key prefixes (`key_i` vs. `bl_key_i`). Election timeouts are randomized (uniform $[3, 6]$ s); exact term sequences may vary between runs but correctness outcomes are deterministic.

C. Experiment Suite

Table II summarizes the six experiments, their fault types, and success criteria. All scripts reside in the `experiments/` directory and are executable from the repository root.

TABLE II
Fault-Injection Experiment Summary

ID	Name	Fault	Success Criterion
E1	Benchmark	None	100% availability; measure latency/throughput
E2	Baseline cmp.	None	Quantify consensus overhead
E3	Leader crash	Container stop	New leader elected ≤ 15 s
E4	Net. partition	Disconnect	No split-brain writes; single leader after heal
E5	Majority loss	2 stops	Cluster unavailable; recovers on restoration
E6	Remediation	Container stop	Watchdog restart ≤ 5 s

VII. Results

We report results in the order defined in Table II, moving from normal operation through progressively severe fault conditions: baseline performance (E1–E2), single-node crash (E3), network partition (E4), majority loss (E5), and automated self-healing (E6).

A. Normal Operation Benchmark (E1)

Under 100 sequential write requests the cluster achieved 100/100 successful writes. Table III reports full performance metrics. The 2.18 messages-per-write confirms that each committed write triggers exactly $N - 1 = 2$ AppendEntries RPCs, one to each follower, consistent with the fan-out expected for a 3-node cluster.

Fig. 4 shows per-request latency over the 100-request run. The distribution is approximately normal with moderate variance; occasional spikes (up to 62 ms) coincide with concurrent heartbeat thread scheduling contention.

TABLE III
NORMAL OPERATION BENCHMARK RESULTS (E1)

Metric	Value
Successful requests	100/100 (100%)
Total time	8.92 s
Throughput	11.21 req/s
Average latency	38.8 ms
Min / Max latency	21.04 ms / 62 ms
Total internal messages	218
Messages per write	2.18

Fig. 4. Per-request write latency during normal operation (E1). The red dashed line indicates the mean (38.8 ms). Latency spikes correlate with concurrent heartbeat thread activity on the leader node.

B. Baseline Comparison (E2)

To quantify the cost of consensus, we compare single-node read latency (GET, no replication required) against quorum write latency (PUT, 2/3 replication required) across 50 requests each. Table IV and Fig. 5 report the results.

TABLE IV
BASELINE COMPARISON: READ VS. QUORUM WRITE (E2)

Op.	Avg	p50	p95	p99
Read	24.3 ms	26.0 ms	28.0 ms	28.9 ms
Write	38.7 ms	40.5 ms	45.7 ms	61.9 ms
Overhead	**14.3 ms**	**14.5 ms**	**17.7 ms**	**33.0 ms**

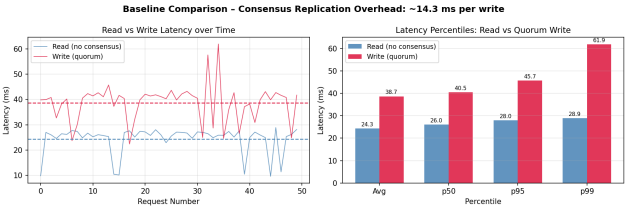

Fig. 5. Baseline comparison (E2). Left: per-request latency for reads (blue) and writes (red) across 50 requests. Right: latency percentile bar chart showing a consistent ∼14.3 ms consensus overhead at average load, widening to ∼33 ms at p99.

The quorum mechanism adds 14.3 ms on average (37% overhead relative to a single-node read) in exchange for fault-tolerance guarantees: the cluster sustains any single-node failure without data loss or availability interruption. The p99 overhead of 33.0 ms reflects occasional write spikes caused by concurrent heartbeat scheduling on the leader thread.

C. Leader Crash and Re-election (E3)

When the leader container was stopped, the two remaining followers detected the absence of heartbeats within their election timeout windows and elected a new leader. The measured total recovery time was 10.15 s, of which ∼8 s is attributable to `docker stop`'s default graceful shutdown period (SIGTERM → SIGKILL after 10 s); the Raft election itself completed within ∼2 s of the container becoming unreachable, consistent with the $[3, 6]$ s configured timeout and the sub-second state propagation via heartbeats.

Post-recovery, the new leader immediately accepted writes, and the previously crashed node rejoined as a follower upon restart, synchronizing its state from the first received AppendEntries snapshot.

D. Network Partition and Split-Brain Prevention (E4)

Fig. 6 illustrates the four-phase timeline of experiment E4, from normal operation through isolation, consistency verification, and network healing.

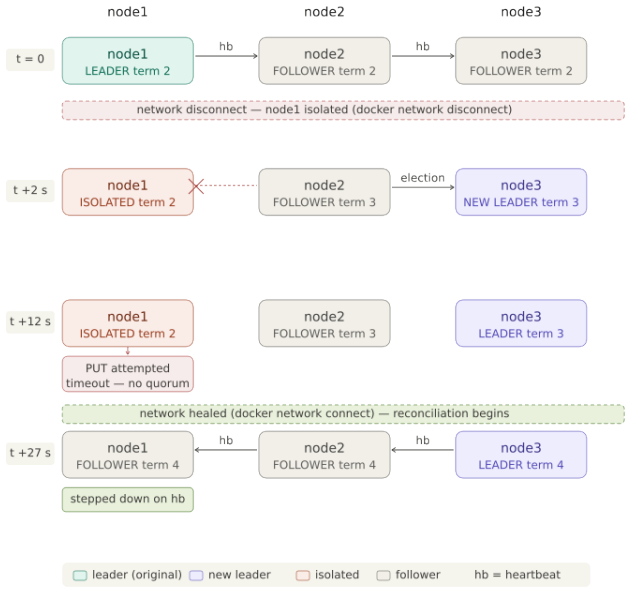

Fig. 6. Network partition experiment (E4) timeline across four snapshots. At $t = 0$, node1 is the active leader (term 2). At $t \approx +2$ s, node1 is disconnected via `docker network disconnect`; node2 and node3 elect node3 as new leader (term 3). At $t \approx +12$ s, a PUT request to the isolated node1 times out, confirming quorum enforcement. At $t \approx +27$ s, the network is healed via `docker network connect`; node1 receives a term-4 heartbeat from node3 and steps down immediately.

Disconnecting the leader's container from the Docker network produced the following sequence: (1) the two reachable followers lost heartbeats and elected a new leader within 2 s (Term +1); (2) the isolated old leader remained locally functional but entered a state where all write replication attempts timed out. **The split-brain write test** confirmed strong consistency: a direct PUT request sent to the isolated leader's

endpoint timed out with a network error before any local commit could be issued, proving that the quorum mechanism blocks divergent writes even when the isolated node believes itself the leader.

After network healing, the first heartbeat received by the old leader (carrying a higher term from the new leader) triggered an immediate step-down. The cluster converged to a single leader within one heartbeat interval (1 s), verified across 15 reconciliation seconds.

E. Majority Loss: Negative Experiment (E5)

Stopping two of three nodes left the surviving node unable to form a quorum. As predicted by Raft safety guarantees, the survivor repeatedly attempted elections (terms 9, 12, 14, ..., 49 across 20 s) but never reached LEADER state. No writes were committed during this period. This confirms the system's CP behavior: it correctly sacrifices availability when a quorum cannot be formed, preventing data inconsistency.

The rapid term increment ($\sim$2 increments/s) is expected Raft liveness behavior: each failed election attempt immediately increments the term and restarts the election timer. Upon restarting the two failed nodes, a leader was elected within 5 s of quorum reformation.

F. Automated Remediation (E6)

The watchdog detected the stopped container within its 2-second polling interval and issued a `docker start` command. The remediated node rejoined the cluster as a follower, loading its persisted store state, and became fully operational within 2 s of the watchdog alert. Total end-to-end remediation time: $\sim$4 s, well within the 5-second success criterion.

VIII. INTEGRATED SYSTEM REQUIREMENTS AND CONFORMANCE SPECIFICATION

Table V presents the integrated system requirements derived from the project specification and the empirical findings of Section VII. Each requirement is assigned a priority level following the convention of [14]: **SHALL** (mandatory), **SHOULD** (strongly recommended), and **MAY** (optional).

Requirements are directly traceable to at least one experiment, metric, or failure-case test, ensuring that every stated property has observable evidence. Table VI summarizes the five conformance checks that verify the most critical safety and availability properties.

Conformance Checks. Table VI maps the five most critical requirements to their verifying experiments and confirms each property holds.

IX. DISCUSSION

Resilience properties verified. All five targeted resilience properties were empirically verified: quorum write consistency, leader re-election, split-brain prevention, correct CP behavior under majority loss, and automated self-healing. The system's behavior is consistent with the theoretical guarantees of Raft's five safety properties [1], particularly Election Safety and State Machine Safety.

Consensus overhead interpretation. The 14.3 ms average overhead of quorum replication over single-node reads is a fundamental cost of strong consistency on a co-located Docker network. In a geographically distributed deployment (inter-datacenter latency 10–100 ms), the overhead would scale proportionally, making the relative cost of consensus significantly larger. The p99 overhead of 33.0 ms (vs. 14.3 ms average) indicates that write tail latency is susceptible to heartbeat thread contention on the single-host deployment, a known limitation of the co-located testbed.

Recovery time composition. The 10.15 s total recovery includes approximately 8 s of Docker's graceful shutdown grace period. Organizations requiring aggressive recovery times should use `docker kill` (immediate SIGKILL) or tune the stop timeout via `docker stop -t 0`; under those conditions, the Raft election of $\sim$2 s would dominate the recovery window.

Message overhead efficiency. The 2.18 measured messages per write (vs. the theoretical $N - 1 = 2$) reflects a small accounting artifact from in-flight heartbeat messages sampled during the benchmark window. The value is consistent with the design and confirms that no superfluous replication occurs.

X. SECURITY, PRIVACY, SAFETY, AND ETHICAL CONSIDERATIONS

Authentication and transport security. The Flask REST API endpoints are unauthenticated and communicate over plaintext HTTP. In a production deployment, all inter-node and client-to-cluster communication must be secured with mutual TLS (mTLS) to prevent eavesdropping, replay attacks, and unauthorized leader impersonation. The `/append_entries` and `/request_vote` endpoints are especially sensitive: an adversary with network access could inject false entries or force term resets without authentication.

Byzantine fault resistance. The implementation assumes crash-stop failures exclusively. A malicious node could violate the Raft protocol by fabricating vote grants, injecting arbitrary log entries, or sending inconsistent AppendEntries responses; the current implementation has no defense against such behavior. Byzantine fault-tolerant variants such as PBFT [8] or HotStuff require $3f + 1$ nodes to tolerate f Byzantine faults; tolerating a single Byzantine actor would require expanding the cluster to at least four nodes.

Watchdog privilege escalation. The watchdog executes Docker CLI commands (`docker inspect`, `docker start`) through the Docker socket, which carries root-equivalent privileges on the host. In a shared environment, this surface should be restricted: named container access control, read-only socket mounts where possible, and audit logging of all watchdog actions.

Data privacy. This system stores only synthetic key-value pairs (`key_i`, `val_i`). No personally identifiable information (PII), health data, location data, or sensitive categories of data are collected, processed, or stored. The persistent store file (`/tmp/raft_store_{node_id}.json`) is written to

TABLE V
INTEGRATED SYSTEM REQUIREMENTS AND CONFORMANCE SPECIFICATION

ID	Requirement	Prio.	Verification Method	Evidence in Experiments
REQ-01	The system **shall** elect a unique leader per term via strict majority vote.	SHALL	Inspect `/status` on all nodes simultaneously; verify single LEADER.	All experiments; E3: single LEADER at Term 4 after crash.
REQ-02	The system **shall** commit write operations only after quorum acknowledgement ($\geq$2/3 nodes).	SHALL	Verify leader waits for 2 AppendEntries ACKs before returning success.	E1: 100/100 commits; E4: isolated write blocked.
REQ-03	The system **shall** elect a replacement leader within 15 s of a leader crash-stop failure.	SHALL	Measure elapsed time from container stop to LEADER in `/status`.	E3: 10.15 s total (~2 s Raft election).
REQ-04	The system **shall** reject write operations when quorum cannot be reached.	SHALL	Attempt PUT to partitioned or minority node; expect 500/timeout.	E4: PUT to isolated leader timed out. E5: no writes committed.
REQ-05	The system **shall** resolve split-brain and converge to a single leader upon network healing.	SHALL	Reconnect partitioned node; observe convergence to a single LEADER term.	E4: all nodes converge to Term 4 within 1 s after heal.
REQ-06	The system **shall** replicate committed data to all available followers via AppendEntries.	SHALL	Verify `store_size` matches across all `/status` endpoints after writes.	E1: store synchronized after 100 writes; E3: rejoined node syncs.
REQ-07	The system **shall** persist committed data to non-volatile storage upon quorum commit.	SHALL	Restart container; verify `Loaded N keys` in startup log.	node.py: `save_store()` called post-commit; load on startup.
REQ-08	The system **shall** maintain full write availability with at most 1 simultaneous node failure.	SHALL	Kill 1 node; verify remaining 2 nodes accept writes via quorum.	E3, E6: cluster continues after single-node loss.
REQ-09	The system **shall** become unavailable for writes when $\geq$2 of 3 nodes fail.	SHALL	Kill 2 nodes; verify survivor never reaches LEADER state.	E5: node1 remains CANDIDATE for full 20-second window.
REQ-10	The system **should** report internal message overhead per write operation.	SHOULD	Sample `internal_messages` counter in `/status` before and after 100 writes.	E1: 218 messages total; 2.18 msg/write (expected $N-1=2$).
REQ-11	The system **should** detect and automatically restart a failed container within 5 s.	SHOULD	Kill container; measure time from stop to FOLLOWER state.	E6: watchdog detects in 2 s; node FOLLOWER within 4 s total.
REQ-12	The system **may** serve read operations from any node without consensus overhead.	MAY	Measure GET latency on follower; compare to PUT latency on leader.	E2: read avg 24.3 ms vs. write avg 38.7 ms (Δ 14.3 ms).

container-local ephemeral storage and is not accessible outside the container namespace without explicit volume mounts.

Operational safety. Fault-injection experiments use container-level operations with bounded blast radius: only the three named containers of this testbed are affected. No production services, external networks, or shared infrastructure are involved.

Ethical considerations. The project does not involve human participants, real infrastructure, or sensitive data. All experimental claims are strictly limited to the evidence collected in the described isolated environment and do not generalize to production deployments or clinically validated systems, consistent with the academic expectations of the project specification [14].

XI. LIMITATIONS AND THREATS TO VALIDITY

Single-host deployment. All three nodes run on the same physical machine, sharing CPU, memory, and a low-latency bridge network ($<$1 ms RTT). Real distributed systems face higher and variable inter-node latencies, which affect election timeout calibration and steady-state throughput. The reported latency figures should be understood as lower bounds for production deployments.

No persistent Raft log. The full-state replication approach means that a simultaneous crash of all three nodes before any heartbeat cycle would result in data loss from the in-memory component (though the JSON file captures the most recently committed quorum state). A production-grade implementation requires persisting individual log entries before applying them to the state machine [1]. The JSON persistence added in this implementation mitigates single-node restarts but does not fully substitute for a WAL.

Non-deterministic election timing. The election timeout is sampled from a uniform distribution, causing variability in recovery time of $\pm$2 s across runs. The reported 10.15 s figure

TABLE VI
CONFORMANCE CHECK RESULTS

CC	Property (REQs)	Evidence	Result
CC-1	Leader uniqueness under partition (REQ-01, REQ-05)	E4: no two nodes in LEADER at same term; isolated node stepped down on receiving Term 4 heartbeat.	**PASS**
CC-2	Quorum enforcement under network fault (REQ-02, REQ-04)	E4: PUT to isolated leader timed out; no local commit issued, quorum replication failed.	**PASS**
CC-3	Recovery time $\leq$15 s (REQ-03)	E3: total 10.15 s; Raft election $\sim$2 s; remaining $\sim$8 s is Docker graceful shutdown.	**PASS**
CC-4	Correct unavailability under majority loss (REQ-09)	E5: survivor in CANDIDATE for full 20 s window; leader elected within 5 s of quorum restoration.	**PASS**
CC-5	Automated remediation $\leq$5 s (REQ-11)	E6: watchdog detected failure in 2 s, restarted container; node rejoined as FOLLOWER within 4 s total.	**PASS**

represents one observed run and should be interpreted as a typical-case measurement rather than a guaranteed bound.

Absence of log compaction. The current implementation transmits the full store on every heartbeat. As store size grows, heartbeat payload size increases linearly, eventually causing write timeouts and spurious leader elections. A log compaction threshold would be required for long-running deployments.

External validity. Results were collected on a single experimental platform. Threats to external validity include differences in OS scheduling, Docker version behavior, and Python thread scheduling across hardware configurations. The Docker Compose configuration and Python version (`python:3.9-slim`) are pinned in the repository to maximize reproducibility.

XII. CONCLUSION AND FUTURE WORK

This paper presented a containerized three-node Raft-based distributed key-value store and its systematic experimental evaluation across six fault scenarios. All targeted resilience properties were empirically verified: the system maintains strong consistency under leader crashes and network partitions, provably blocks split-brain writes via quorum enforcement, correctly becomes unavailable under majority loss (CP behavior), and recovers automatically via a watchdog within 4 s. A quantitative baseline comparison establishes a 14.3 ms consensus overhead at average load and 33.0 ms at the 99th percentile, representing a measurable cost for fault-tolerant writes, with tail latency sensitive to heartbeat thread contention on the single-host testbed. The message-overhead metric confirms the expected $N - 1 = 2$ replication fan-out per committed write.

Future work should address: (i) replacement of full-state replication with incremental Raft log entries and snapshot transfer as specified in [1]; (ii) Byzantine fault tolerance using PBFT or threshold signature schemes; (iii) multi-Raft configurations for horizontal write scaling; (iv) geographically distributed deployment to characterize real-world latency overhead; and (v) integration with a Chaos Mesh [13] framework for automated, large-scale fault campaigns beyond manual script execution.

REFERENCES

[1] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proc. USENIX Annual Technical Conf. (ATC)*, Philadelphia, PA, USA, Jun. 2014, pp. 305–319.

[2] L. Lamport, "Paxos made simple," *ACM SIGACT News*, vol. 32, no. 4, pp. 18–25, Dec. 2001.

[3] E. Brewer, "Towards robust distributed systems," in *Proc. 19th ACM Symp. Principles of Distributed Computing (PODC)*, Portland, OR, USA, Jul. 2000, p. 7.

[4] S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, Jun. 2002.

[5] M. J. Fischer, N. A. Lynch, and M. S. Paterson, "Impossibility of distributed consensus with one faulty process," *J. ACM*, vol. 32, no. 2, pp. 374–382, Apr. 1985.

[6] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed, "ZooKeeper: Wait-free coordination for internet-scale systems," in *Proc. USENIX Annual Technical Conf. (ATC)*, Boston, MA, USA, Jun. 2010, pp. 145–158.

[7] J. C. Corbett *et al.*, "Spanner: Google's globally distributed database," *ACM Trans. Comput. Syst.*, vol. 31, no. 3, pp. 1–22, Aug. 2013.

[8] L. Lamport, R. Shostak, and M. Pease, "The Byzantine generals problem," *ACM Trans. Program. Lang. Syst.*, vol. 4, no. 3, pp. 382–401, Jul. 1982.

[9] T. D. Chandra and S. Toueg, "Unreliable failure detectors for reliable distributed systems," *J. ACM*, vol. 43, no. 2, pp. 225–267, Mar. 1996.

[10] A. Basiri *et al.*, "Chaos engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, May/Jun. 2016.

[11] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media, 2017.

[12] etcd Maintainers, "etcd: A distributed, reliable key-value store for the most critical data of a distributed system," Cloud Native Computing Foundation (CNCF), 2013–present. [Online]. Available: https://etcd.io

[13] Chaos Mesh Authors, "Chaos Mesh: A Powerful Chaos Engineering Platform for Kubernetes," CNCF Sandbox Project, 2020. [Online]. Available: https://chaos-mesh.org

[14] *Project Specification: Resilience in Distributed Systems (Project 43, Tier A)*, Faculty of Computer Science, Alexandru Ioan Cuza University of Iași, 2025.